

Minishell — Roadmap duo

Plan de bataille sur 40 jours, à partir de la base ShellPy

Départ J1 = jeu 30 juil 2026 Code gelé J30 = ven 28 août Blackhole ≈ J40 = lun 7 sept Rythme 3–4 h/jour × 2

Répartition parsing / exécution

1 Calibrage — le calcul qui commande tout le reste

40 jours de calendrier ne sont pas 40 jours de code. Le blackhole se referme quand le projet est **validé**, pas quand il est fini : il faut caler 3 soutenances avec des pairs disponibles, puis laisser passer Deepthought. Compter 7 à 10 jours.

Deadline réel du code	J30 (ven 28 août) — pas J40
Budget disponible	30 jours × 3,5 h × 2 personnes ≈ 210 h de duo
Coût réel d'un mandatory propre	≈ 200–220 h en duo
Verdict	Faisable, marge nulle.

LES TROIS CONSÉQUENCES À ACCEPTER DÈS AUJOURD'HUI

- Aucun gros refactor n'est finançable après J16. Le design des quotes doit être juste du premier coup — c'est toute la raison d'être de la Phase 0.
- Le bonus n'est décidé qu'au Gate J23, jamais avant. Un mandatory imparfait annule intégralement le bonus (le sujet est explicite).
- Une semaine perdue se paie sur la Phase 4 (chasse aux fuites et aux crashes), c'est-à-dire au pire endroit possible : un minishell complet qui fuit ou segfault prend 0.

2 Décisions gelées — signées à deux, non rediscutables

DÉCISION	CHOIX	POURQUOI
Suivi des quotes	Le token est une struct, pas une chaîne : une suite de morceaux, chacun portant son type de quote (aucune / simple / double).	Le lexer est le seul endroit qui voit jamais ' et ". Ce qu'il ne note pas est perdu pour toujours. C'est ce qui permet de distinguer "\$A" (1 arg) de \$A (n args). Cohérent avec votre méthode « on stocke dans une struct ».
Forme du parser	Arbre avec un champ discriminant (commande / pipeline / futur &&).	Quelques heures maintenant, plusieurs jours si conversion tardive. Votre champ "type" Python est déjà ce discriminant.
Arguments	Un seul tableau, élément 0 = la commande. Pas de champ cmd séparé.	Supprime d'un coup le bug grep grep f (un argument disparaît) et le cas tordu « redirection avant la commande ».
Redirections	Liste ordonnée, chaque élément portant son type (< > >> <<) et sa cible.	Bash les applique toutes, de gauche à droite. > a > b crée les deux fichiers. Deux slots écrasables rendent aussi le heredoc impossible.
Environnement	Une seule table, propriétaire unique, passée en paramètre.	La règle « une seule globale, contenant uniquement le numéro de signal » interdit de la mettre en global. Deux espaces de noms séparés se désynchronisent.
Reconnaissance des opérateurs	Dans l'automate du lexer, caractère par caractère. Aucun pré-traitement de la ligne.	Un remplacement global sur la ligne brute ne sait pas s'il est dans une quote : il corrompt les données (echo "a b" devient a b).

LIVRABLE N°1 DE LA PHASE 0

Le contrat écrit des **trois structs** — token, commande, nœud — avec pour chacune : ses champs, **son propriétaire**, **qui la libère**, et sa durée de vie. Voir §7. Rien d'autre ne démarre avant.

3 Héritage ShellPy — ce qui traverse le portage

Constat de calibrage : **le parsing n'est pas à 60 %, il est à ~15 %**. La ligne `shlex.split(line)` fait à elle seule les trois morceaux les plus durs du projet — reconnaissance des quotes, découpage en mots, suppression des quotes. `shlex` n'existe pas en C : 36 lignes de Python cachent 350 à 450 lignes de C normées. Bonne nouvelle : les ~200 lignes hors-sujet à supprimer compensent, la roadmap tient.

ON GARDE

- Le découpage en modules `parsing` / `execution` / `builtins` / `utils` — aide aussi pour la Norme
- `shell_state` : un état central passé partout — architecture obligatoire ici
- `handle_redir` isolé dans son propre fichier
- La séparation `parse_simple` / `parse_pipeline` — c'est déjà une descente récursive
- La silhouette du dict de commande, et son champ `type`
- L'ordre lexer → parser → expansion → exécution

ON REFAIT

- **Le lexer entièrement** : automate à états, zéro pré-traitement de la ligne
- **La place de l'expansion** : elle doit venir *après* le découpage, avec l'info de quote intacte. La suppression des quotes est la **dernière** étape
- Le tag `__NO_EXPAND__` → remplacé par un champ de struct
- Reconstruction de `argv` par comparaison de chaînes → parcours unique gauche→droite
- Slots `stdin/stdout` uniques → liste ordonnée
- Messages d'erreur français → format bash, sur **`stderr`**

ON SUPPRIME

- `func_if.py` (123 l.) — hors sujet
- `func_for.py` (54 l.) — hors sujet
- `;` — le sujet interdit de l'interpréter
- `&`, `background.py`, la table de processus en fond
- `help`
- `user_vars` + `is_affect` — une seule table d'env suffit

Absents du Python, à écrire de zéro : `<<` `heredoc`, `export`, `env`, `pwd`, `echo -n`, la gestion des signaux, les codes de sortie conformes.

4 Les cinq phases

0 Le contrat — à deux, aucun code séparé

J1–J2 · 30–31 juil

La phase que tout le monde saute et qui coûte une semaine plus tard. **Vous travaillez ensemble, sur le même écran.**

- Écrire le contrat des trois structs (§7) : champs, propriétaire, libération, durée de vie.
- Définir le mécanisme de suivi des quotes dans le détail — le seul point où la base Python ne vous donne rien.
- Trancher qui possède la table d'environnement (`export` , `unset` , l'expansion y touchent tous les trois).
- Découper explicitement le heredoc : qui le lit, qui branche le descripteur.
- Repo git, stratégie de branches, **un seul propriétaire du Makefile**, norminette lancée dès le premier fichier.
- Écrire à la main ~25 cas de test tirés de bash réel : entrée → sortie attendue → `$?` attendu. C'est votre spec exécutable, et elle servira en soutenance.

GATE J2 — Vous savez tous les deux dessiner les trois structs au tableau, à l'identique, sans regarder. Le fichier de tests existe.

1 Deux squelettes en parallèle, avec un mock au milieu

J3–J9 · 1–7 août

TOI — PARSING

- Boucle readline + historique
- L'automate à états du lexer : gestion des quotes, info de quote **conservée** par morceau
- Production de tokens affichables pour vérification

BINÔME — EXÉCUTION

- `fork` / `execve` / résolution du PATH
- Sur une structure de commande **écrite en dur dans le code**
- Une seule commande, aucun pipe

LE POINT CRITIQUE DE TOUTE LA ROADMAP

Il ne doit **surtout pas** attendre ton parser. Il fabrique une fausse commande à la main et travaille dessus. Deux moitiés qui se compilent séparément contre une interface figée avancent en parallèle ; deux moitiés qui se compilent ensemble avancent en série. C'est ce mock qui vous fait gagner une semaine.

GATE J9 — `ls -la` s'exécute de bout en bout depuis une struct codée en dur, et ton lexer sort les bons tokens sur 15 lignes de test.

2 Première jonction

J10–J16 · 8–14 août

TOI — PARSING

- Le parser : tokens → structs de commande
- Détection des erreurs de syntaxe, `exit 2`
- Abandon propre en cours de construction : libérer un arbre à moitié bâti

BINÔME — EXÉCUTION

- Pipes à n commandes
- Redirections `<` `>` `>>` , appliquées **dans l'ordre**
- Hygiène des descripteurs, boucle de `waitpid` , `$?`

Vers J13 : la première vraie jonction — ton parser alimente son exécuter. Prévois 1 à 2 jours de frottement. **C'est planifié, ce n'est pas un échec.**

GATE J16 — `cat f | grep x > out` fonctionne depuis un vrai parsing. Si tu es encore sur le lexer ici : ~1 semaine de retard, bonus abandonné définitivement.

3 Les morceaux durs

J17–J23 · 15–21 août

TOI — PARSING

- Expansion de `$VAR` et `$?`
- Word-splitting **uniquement** sur le résultat non quoté
- Suppression des quotes, en toute dernière étape
- Parsing du heredoc, expansion dedans selon que le délimiteur était quoté ou non

BINÔME — EXÉCUTION

- Les 7 builtins, stockage de l'env
- Signaux : interactif, enfant, et pendant un heredoc
- Double chemin : builtin seul → dans le père ; builtin dans un pipe → dans le fils
- Application des redirections **commune** aux builtins et aux externes

GATE J23 — Toute la liste du mandatory passe sur votre fichier de tests. **DÉCISION BONUS ICI, ET UNIQUEMENT ICI.** Si tu refactorises encore les quotes à ce stade : zone rouge, stop features, on stabilise.

4 Chasse — ce n'est pas du remboursement

J24–J30 · 22–28 août

Les fonctionnalités ne rattrapent jamais un crash ni une fuite. C'est la phase qui décide de la note.

- `valgrind --leak-check=full --track-fds=yes --trace-children=yes` sur les 25 cas de test.
- Norminette clean sur 100 % des fichiers, y compris ceux du bonus s'il existe.
- **Relecture croisée complète** : chacun lit l'intégralité de la moitié de l'autre. Non négociable — vous êtes interrogés tous les deux, et la modification demandée peut tomber sur n'importe quelle moitié.
- Diff systématique contre bash sur les cas limites.
- README (voir §8).
- **Réserver les soutenances dès J28**, pas à J31.

GATE J30 — Code gelé et poussé. Aucune fonctionnalité ajoutée après cette date.

5 Soutenances + Deepthought

J31–J40 · 29 août – 7 sept

Fenêtre entièrement consacrée à la validation. Trois soutenances à caler, Deepthought derrière. C'est toute la raison pour laquelle le code est fini à J30 et pas à J38.

OBJECTIF — Projet validé avant la fermeture du blackhole.

5 Répartition et territoires partagés

La charge n'est pas 50/50 : le parsing représente ~60 % du temps, l'exécution est plus courte mais bien plus dense en bugs vicieux. Le déséquilibre se corrige en donnant à l'exécution tout ce qui l'entoure.

ZONE	PROPRIÉTAIRE	NOTE
Lexer, parser, expansion	Toi	Le gros morceau. ~60 % du budget total.
fork / exec / pipes / redirections	Binôme	Court mais piégeux : les fuites de descripteurs bloquent le shell sans message d'erreur.
Builtins (7)	Binôme	Rééquilibrage de charge.
Signaux	Binôme	Une seule globale autorisée, contenant uniquement le numéro de signal.
Table d'environnement	Partagé	Propriétaire à désigner en Phase 0. Ton expansion la lit, ses builtins l'écrivent.
Heredoc	Partagé	Toi : détection et lecture. Lui : branchement du descripteur. À découper en Phase 0, pas au moment d'y arriver.
Codes de sortie	Partagé	Ton parser produit le 2 (syntaxe), son exécuteur les 126/127/130/131.
Makefile, libft	Un seul de vous deux	Deux propriétaires = conflits de merge permanents.

6 Signaux d'alarme — à vérifier à chaque gate

SYMPTÔME	DIAGNOSTIC	RÉACTION IMMÉDIATE
À J9, l'exécution attend ton parser	Le mock n'a pas été fait — planning cassé, vous êtes en série	Il code la struct en dur aujourd'hui, avant toute autre chose
À J16, tu es encore sur le lexer	≈ 1 semaine de retard	Bonus abandonné définitivement. On protège la Phase 4.
À J23, tu refactorises les quotes	Zone rouge — le design de Phase 0 était faux	Stop features. On stabilise ce qui marche déjà.
Le shell se bloque sans message sur un pipe	Descripteur non fermé : le lecteur ne voit jamais l'EOF	Audit systématique : chaque processus ferme les deux extrémités qu'il n'utilise pas
Vous n'avez pas relu le code de l'autre à J28	Risque direct en soutenance	Une demi-journée chacun, toutes affaires cessantes
Vous ajoutez une fonctionnalité après J30	Le code n'est pas gelé	Refuser. Le sujet ne demande rien d'autre que le mandatory.

7 Le contrat des trois structs — livrable de Phase 0

Pour chacune, la question qui compte n'est pas « quels champs » mais « **qui la libère** ».

STRUCT	PRODUITE PAR → CONSOMMÉE PAR	DOIT PORTER	DURÉE DE VIE
Token	Lexer → parser	Sa nature (mot ou opérateur, et lequel). Pour un mot : ses morceaux, chacun avec son type de quote.	La plus courte. Meurt dès que le parser a fini.

Commande	Parser → exécuteur	Un tableau d'arguments (élément 0 = la commande) et une liste ordonnée de redirections typées.	Jusqu'à la fin de l'exécution de la ligne.
Nœud	Parser → exécuteur	Un discriminant, et ses enfants. C'est ce qui relie les commandes entre elles.	Idem commande. Se libère récursivement.
État du shell	Vit à côté de l'arbre	La table d'environnement, le dernier code de sortie. Passé en paramètre, jamais en global.	Jusqu'à <code>exit</code> .

LE PIÈGE QUE PYTHON VOUS CACHAIT

Un dict Python disparaît tout seul. En C, **chaque struct allouée est une fuite en attente**. Le réflexe à prendre dès la Phase 0 : **pour chaque struct définie, écrire sa fonction de libération dans la même heure**. Trois raisons — une struct imbriquée se libère récursivement et on oublie toujours un niveau si on le fait après coup ; ton parser doit pouvoir abandonner sur une erreur de syntaxe et libérer un arbre à moitié construit ; et ça force à trancher tout de suite qui possède les chaînes de caractères.

Ne jamais faire pointer une commande vers du texte appartenant à un token : leurs durées de vie diffèrent, c'est la source de double-free la plus fréquente du projet.

8 Checklist avant de geler le code

DOMAINE	À VÉRIFIER
Codes de sortie	127 introuvable · 126 permission ou répertoire · 1 erreur de redirection · 2 erreur de syntaxe · 130 ctrl-C · 131 ctrl-\ · <code>exit</code> masqué modulo 256 · <code>exit abc</code> ne crashe pas
Messages d'erreur	En anglais, format bash, sur stderr — pas sur stdout, les pairs comparent pendant l'éval
Signaux interactifs	ctrl-C → nouveau prompt sur une nouvelle ligne · ctrl-D → quitte · ctrl-\ → rien · comportement différent pendant un heredoc
Quotes	<code>echo "a b"</code> · <code>echo ">"</code> · <code>a"b c"d</code> = un seul mot · <code>'\$A'</code> non expansé · <code>"\$A"</code> = 1 arg vs <code>\$A</code> = n args · quote non fermée → erreur de syntaxe
Redirections	<code>> a > b</code> crée les deux · <code>< f1 < f2</code> ouvre les deux · <code>> out echo hi</code> fonctionne · plusieurs heredocs dans un pipeline
Builtins	<code>cd</code> et <code>export</code> seuls persistent · dans un pipe, ils ne modifient pas le shell · <code>pwd > f</code> écrit bien dans le fichier
Mémoire	valgrind clean sur les 25 cas, descripteurs inclus. Les fuites de <code>readline</code> sont tolérées ; les vôtres, non.
Norme	25 lignes par fonction · 5 fonctions par fichier · 4 paramètres · bonus inclus dans le check
README	Première ligne en italique au format imposé · sections Description, Instructions, Resources · description de l'usage de l'IA · rédigé en anglais
Makefile	<code>\$(NAME) all clean fclean re</code> · <code>-Wall -Wextra -Werror</code> · pas de relink inutile · compile libft via son propre Makefile